A PROJECT REPORT
ON

"URBAN PROPERTY MANAGEMENT SYSTEM"

Submitted to
UNIVERSITY OF PUNE

In Partial Fulfilment of the Requirement for the Award of

BACHELOR'S DEGREE IN
COMPUTER ENGINEERING

BY

Asmita Narendra Jagtap (13)
Ketaki Ganpat Kale (15)

UNDER THE GUIDANCE OF

PROF. P. Biswas

DEPARTMENT OF COMPUTER ENGINEERING

SMES Sanghavi College of Engineering

AFFILIATED TO
UNIVERSITY OF PUNE

SMES Sanghavi College of Engineering
Department of Computer Engineering

Shree Mahavir
Education Society's

Sanghavi
College of Engineering
Nashik

CERTIFICATE

This is certify that the project entitled

"URBAN PROPERTY MANAGEMENT SYSTEM"

submitted by

Asmita Narendra Jagtap (13)
Ketaki Ganpat Kale (15)

is a record of bonafide work carried out by them, in the partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Engineering) at Sanghavi College of Engineering, Pune under the University of Pune. This work is done during year 2025-2026, under our guidance.

Date: 31/10/2025

Prof .P.Biswas	Prof. P.Biswas	Prof.Mohammad Ali	
Project Guide	HOD, Computer Department	Principal	External Examiner

Acknowledgements

We are profoundly grateful to Prof. P. Biswas for his expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion.

We would like to express deepest appreciation towards Prof. Mohammad Ali, Principal, Sanghavi college of Engineering, Prof. P. Biswas, Head of Department of Computer Engineering and Prof. P. Biswas, Project Coordinator whose invaluable guidance supported us in completing this project.

At last we must express our sincere heartfelt gratitude to all the staff members of Computer Engineering Department who helped me directly or indirectly during this course of work.

Asmita Narendra Jagtap
Ketaki Ganpat Kale

ABSTRACT

The Urban Property Management System is a web-based application designed to streamline and automate property administration in urban residential buildings. Modern housing societies consist of multiple stakeholders such as building owners, building managers, flat owners, and tenants, each requiring tailored access to property-related information. Traditional manual processes for managing flats, ownership records, tenant details, and monthly bill payments are time-consuming, prone to errors, and lack transparency.

This system introduces a role-based access model where each user accesses the platform through secure login credentials with privileges aligned to their authority. The building owner can monitor all flats, owners, and tenant information. The building manager can view flat sales, ownership history, and building occupancy. Flat owners can manage tenant information, generate and send monthly charges such as water and electricity bills, and monitor payment status. Tenants can easily pay bills online, and payment notifications are instantly shared with the corresponding flat owner.

The system enhances communication, ensures accuracy in record-keeping, and promotes a transparent digital property management process. It reduces paperwork, improves efficiency, and provides a structured and scalable platform for modern housing management. Future enhancements may include mobile app integration, online maintenance request handling, digital agreements, and automated reminders to further enhance usability and convenience.

Index

Sr No.	Name of Topic	Page no.
1	Introduction 1.1. Background 1.2. Problem Statement	1
2	Literature Survey	3
3	Software Requirement Specification(Flowchart, System Architecture)	4
4	Requirement Analysis	5
5	System Design	6
6	System Testing	8
7	Project Planning	10
8	Implementation	11
9	Screenshots of Project	16
10	10.1 Future Scope 10.2 Conclusion	17

Index of Figure

Sr. No	Name of Figure	Page. No.
3.2	Flow chart	4
5.1	System Architecture	7

Index of Table

Sr. No	Name of Figure	Page. No.
3.1	Software Requirement	4
5.1	Test Cases	8,9

Introduction

Urban residential properties today involve complex structures consisting of multiple flats, owners, tenants, and maintenance responsibilities. As cities grow rapidly, residential building management has evolved into a multi-tiered system where efficient coordination between building owners, managers, flat owners, and tenants is essential. However, most property management activities are still carried out manually through paperwork, spreadsheets, or fragmented communication methods such as phone calls and messages. This often leads to issues such as miscommunication, loss of records, delayed payments, and difficulty in tracking resident information.

The **Urban Property Management System** is developed to address these challenges by providing a digital platform that centralizes and automates key operations related to property administration. The system offers **role-based access**, ensuring each user interacts with the platform according to their responsibilities. Building owners can view complete building and flat information, building managers can track flat sale status and ownership records, flat owners can manage tenant details and send utility bills, while tenants can conveniently pay bills online and receive notifications.

By providing a secure and structured digital workflow, this system aims to reduce manual effort, enhance transparency, and improve communication across all levels of building management. The platform not only ensures organized storage of property data but also strengthens accountability and simplifies routine tasks such as billing, tenant management, and payment tracking. With scope for future expansion into mobile applications, maintenance service tracking, and automated alerts, this system positions itself as a scalable solution for modern property administration in urban environments.

1.1 Background

Urbanization has significantly increased the demand for organized residential complexes and housing societies. In multi-storey buildings and gated communities, property ownership and tenant management involve multiple stakeholders, including building owners, society managers, flat owners, and tenants. Traditionally, essential tasks such as maintaining flat ownership records, tenant details, bill management, rent tracking, and communication were handled manually through registers, paper notices, and informal phone communication.

These outdated practices often result in data loss, miscommunication, delayed responses, and difficulty in accessing accurate records. With the increase in digital adaptation across industries, property management has also shifted toward automated solutions. Modern cloud-based and mobile systems offer faster communication, better record-keeping, and improved convenience for both property owners and tenants.

1.2 Problem Statement

Managing residential properties in urban environments involves multiple stakeholders such as building owners, managers, flat owners, and tenants. Traditional manual methods like paper records, spreadsheets, and informal communication lead to difficulties in maintaining accurate data, tracking ownership and tenant details, generating bills, monitoring payments, and ensuring timely communication.

These systems are prone to human error, lack transparency, and consume significant time and effort. There is a need for a centralized digital platform that provides role-based access, automates day-to-day property management tasks, and enables efficient communication between all users to ensure smooth and transparent property administration.

2. Literature Survey

Property management systems have evolved with the increasing need for efficient, transparent, and automated real-estate administration. Early systems primarily focused on manual book-keeping and physical record storage, which made updates slow and error-prone. With the rise of digital technology, researchers and developers explored software-based solutions that offer centralized data storage and streamline property-related operations.

Several studies and commercial applications highlight the adoption of database-driven property management systems for tasks such as tenant record maintenance, rent tracking, and communication between owners and tenants. Most existing systems are built for large real-estate companies or commercial property management, offering features like accounting, lease management, and automated notifications. However, many lack customized support for multi-level residential building structures seen in urban environments, where different users — building owners, managers, flat owners, and tenants — require distinct privileges and system access.

Research work in smart housing and digital governance emphasizes secure authentication, user-specific dashboards, and online payment mechanisms as essential features in property automation. Modern systems also integrate mobile-based services and cloud storage to provide accessibility and scalability. Despite these advancements, many traditional residential societies still rely on offline practices due to lack of technical awareness and limited access to tailored digital tools.

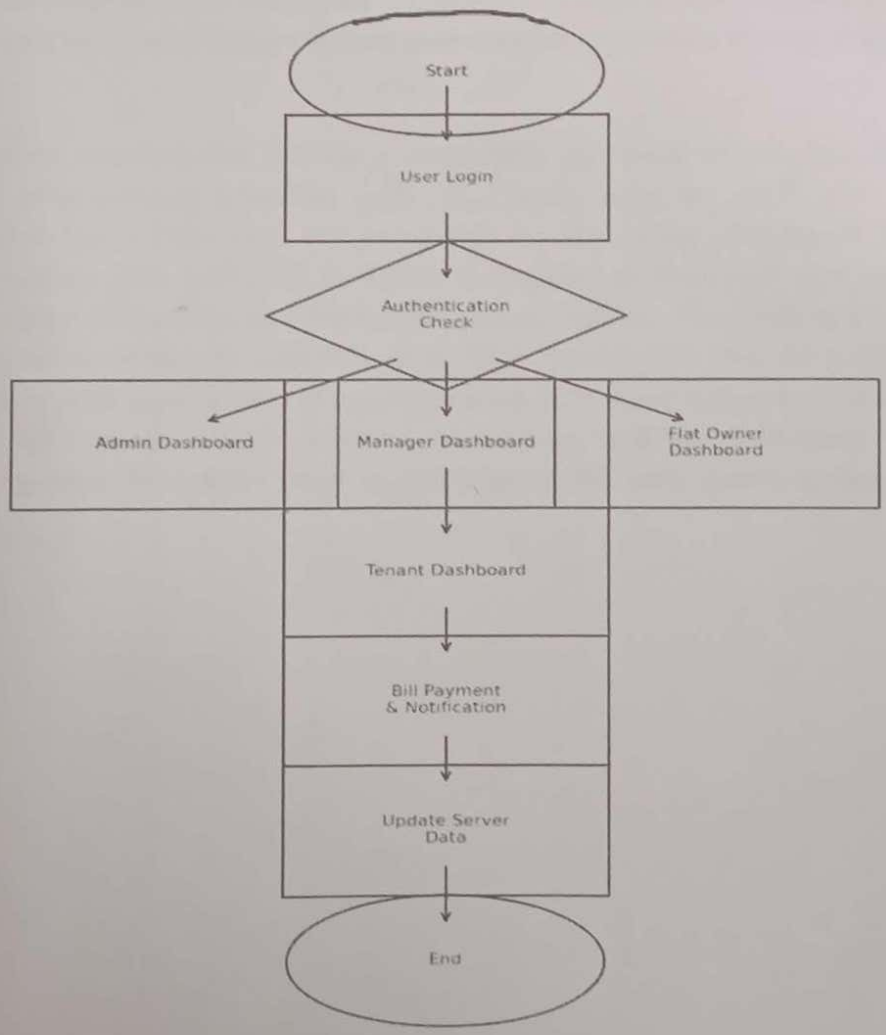
The Urban Property Management System bridges this gap by providing a role-based, secure, and user-friendly online platform designed specifically for layered residential property management. It combines features such as ownership tracking, tenant management, online bill generation, payment monitoring, and real-time communication, contributing to efficient and transparent urban property administration.

3. Software Required Specification

3.1 Software Requirements

Category	Requirement
Operating System	Windows
Frontend Technologies	React JS , Material UI
Backend Technology	Node.js
Code Editor	Vs Code
Browser	Goggle chrome / Microsoft Edge

3.2 System Flowchart



4. Requirement Analysis

The requirement analysis phase focuses on identifying and understanding the needs of the proposed User Monitoring System. The system must provide efficient monitoring of multiple client systems connected over a network. Functionally, it should allow the administrator to track user activities, such as application usage, website access, keystrokes, and file transfers. The admin must be able to view real-time screen activity and receive notifications in case of suspicious or unauthorized actions. To ensure restricted access, a secure login mechanism is required, and all monitored data should be stored safely in the server database for future analysis. The system may also support remote control features like shutdown or restart of client systems to maintain workplace discipline and prevent misuse.

In addition to functional needs, the system has several non-functional requirements. It must operate in real-time with minimum delay and provide reliable communication between server and client systems. The interface should be simple, user-friendly, and efficient for administrators. Data security is crucial, so the system should incorporate secure authentication and encrypted log storage. It should also be scalable to support multiple user systems without affecting performance and should be easy to maintain and update.

The software requirements include a compatible operating system like Windows or Linux, a programming language such as Python, Java, or .NET, and supporting technologies like HTML, CSS, and JavaScript for the admin dashboard interface. A database system such as MySQL or SQLite is required to store logs, and LAN or Wi-Fi network connectivity must be available for communication. The hardware requirement includes a server computer with at least an Intel i5 processor, 8GB RAM, and sufficient storage along with basic client systems connected in the same network environment. The users of the system expect seamless monitoring without disturbing their usage experience, while the admin requires quick alerts and easy access to system control features.

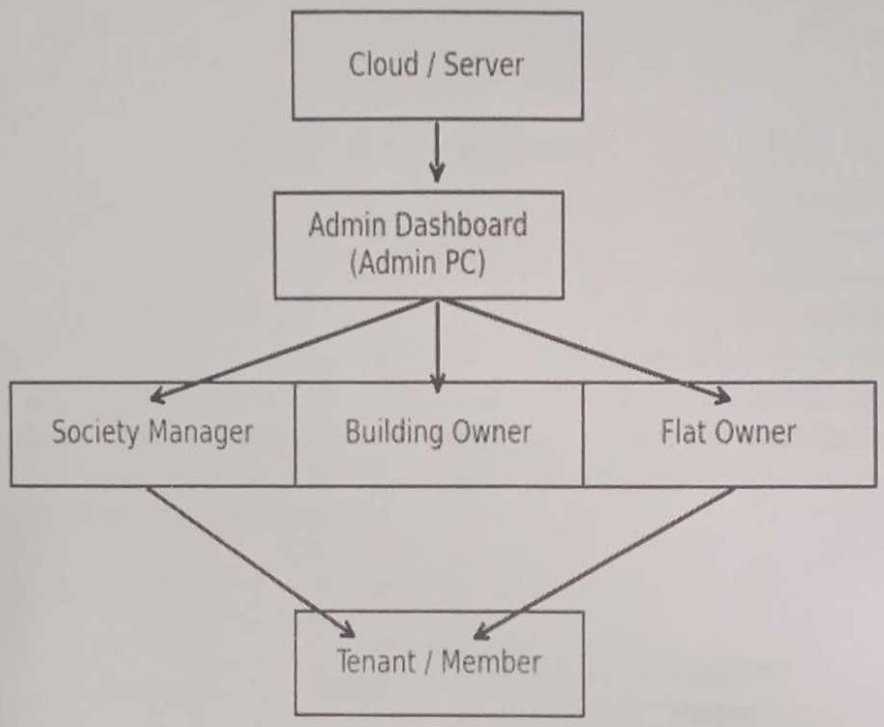
5. System Design

The system design phase focuses on transforming the project requirements into a structured solution blueprint. The Urban Property Management System follows a multi-tier architecture that ensures secure and efficient interaction between different user roles such as the Building Owner (admin), Building Manager, Flat Owner, and Tenant. Each user has a dedicated login panel and role-based access control to ensure that sensitive property and tenancy information is accessed only by authorized users.

The building owner, being the highest-level authority, can manage building details, view flat allocations, monitor tenants, and handle billing information. The building manager assists in daily operational management and has controlled access to flat and occupancy data. The flat owner handles individual flat-related activities such as monitoring tenant details, generating water and electricity bills, and sending them digitally to tenants. Tenants can log in to view and pay bills securely and can send payment confirmations, which are reflected to the flat owners through notifications or system updates. All these interactions are centrally processed and stored in the system server, ensuring data consistency and availability.

From a technical standpoint, the system is designed using a modular architecture to ensure maintainability and scalability. Key modules include authentication, property management, billing and payment module, tenant management, and notification module. The backend database handles user credentials, property records, billing logs, payment information, and message logs. Communication between the user interface and database is managed through server-side scripts and structured queries. The system ensures data security through access validation and controlled module permissions. This design enables efficient management of urban residential properties while offering convenience, transparency, and digital automation to stakeholders.

5.1 System Architecture



6. System Testing

System testing is a critical phase in the software development life cycle that ensures the entire Urban Property Management System works as expected and meets the defined requirements. In this project, system testing was performed after unit and integration testing to validate the complete functionality of all user roles, including Building Owner, Manager, Flat Owner, and Tenant. The goal of testing was to verify that each module — such as user authentication, property and tenant management, billing system, notification system, and payment gateway — functions correctly and seamlessly interacts with other modules.

Both functional and non-functional testing approaches were applied. Functional testing verified key operations like user registration and login, flat assignment, bill generation by flat owner, payment submission by tenants, and notification delivery. Non-functional testing focused on performance, usability, security, and compatibility. Security testing ensured only authorized users accessed their privileges based on hierarchy levels. Usability testing confirmed that user interfaces were simple and intuitive for all categories of users. Performance testing was conducted to measure system stability under multiple user requests. The system successfully passed all major test scenarios ensuring that it performs efficiently, maintains data accuracy, and delivers a reliable user experience.

6.1 Test Cases Table

Test Case ID	Description	Input	Expected Output	Status
TC01	Verify Tenant login with valid credentials	Correct username & password	Tenant dashboard opens	Pass
TC02	Verify Tenant login with invalid credentials	Wrong username/password	Error message displayed	Pass
TC03	Verify Tenant can view flat details	Click "View Flat Info"	Flat details displayed	Pass
TC04	Verify Tenant can view owner details	Click "View Owner Info"	Owner details displayed	Pass
TC05	Verify Flat Owner login with valid credentials	Correct username & password	Flat Owner dashboard opens	Pass

Test Case ID	Description	Input	Expected Output	Status
TC06	Verify Flat Owner login with invalid credentials	Wrong username/password	Error message displayed	Pass
TC07	Verify Flat Owner can view tenant details	Select "My Tenants"	Tenant details displayed	Pass
TC08	Verify Flat Owner can view flat details	Click "View Flat Info"	Flat information displayed	Pass
TC09	Verify Manager login with valid credentials	Correct username & password	Manager dashboard opens	Pass
TC10	Verify Manager login with invalid credentials	Wrong username/password	Error message displayed	Pass
TC11	Verify Manager can view all flats	Click "View Flats"	Flats list displayed	Pass
TC12	Verify Manager can view owners & tenants	Click "View Owner/Tenant List"	Owner & tenant list displayed	Pass
TC13	Verify unauthorized access prevention	Tenant tries to access Owner/Manager pages	Access denied message	Pass
TC14	Verify Logout function	Click "Logout"	User returned to login screen	Pass

7 Project Planning

Project planning is a crucial phase in the software development life cycle where the overall structure and direction of the project are determined. In this stage, the project objectives, scope, and deliverables are defined to ensure clarity and smooth execution. The primary objective of the project is to develop a secure and user-friendly system with essential functionalities such as tenant login, flat owner login, and manager login. The system aims to improve efficiency, accuracy, and accessibility by automating manual processes and providing a centralized platform for users.

The scope of the project includes requirement gathering, system design, database creation, interface development, backend coding, security implementation, and system testing. The **Waterfall development model** is followed where each phase—requirement analysis, system design, implementation, testing, deployment, and documentation—progresses sequentially. A planned work breakdown structure divides the tasks into modules such as UI design, backend development, database setup, and testing, each assigned a specific duration to ensure timely completion.

Resource planning includes the use of a laptop/desktop, development tools like Visual Studio Code, and documentation tools such as MS Word and draw.io for diagrams. Risk analysis is also performed to identify possible risks like delay in development, technical challenges, or data loss, and strategies are prepared to mitigate them through regular backups, proper documentation, and systematic coding practices. The successful completion of this planning phase ensures that the project proceeds in an organized manner and results in a functional, efficient, and secure multi-user login system.

8 Implementation

Modules

The Urban Property Management System is divided into multiple modules to ensure smooth and systematic workflow. In this partial phase of development, the system primarily focuses on the authentication and access control modules designed for three user types: Tenant, Flat Owner, and Manager. Each module has specific functions and access privileges according to the user's role, ensuring a role-based secure system structure.

Tenant Login Module

The Tenant Login module allows tenants to securely access the system using their registered credentials.After successful login, tenants can view basic information such as their personal profile and billing details (future scope includes bill payment and complaints). Incorrect credentials result in a login error message, ensuring secure access. This module ensures that only authorized tenants can access their account.

Flat Owner Login Module

This module enables flat owners to log into the system using valid login details. Flat owners can view details related to their tenants and building association (limited as this is the half-project stage). The system verifies the owner's credentials and grants dashboard access only on successful validation.

Manager Login Module

The Manager Login module grants access to the building manager. Once logged in, the manager can view flat ownership and basic building-related data. Since this is a development-phase version, the module mainly includes login authentication and dashboard access.

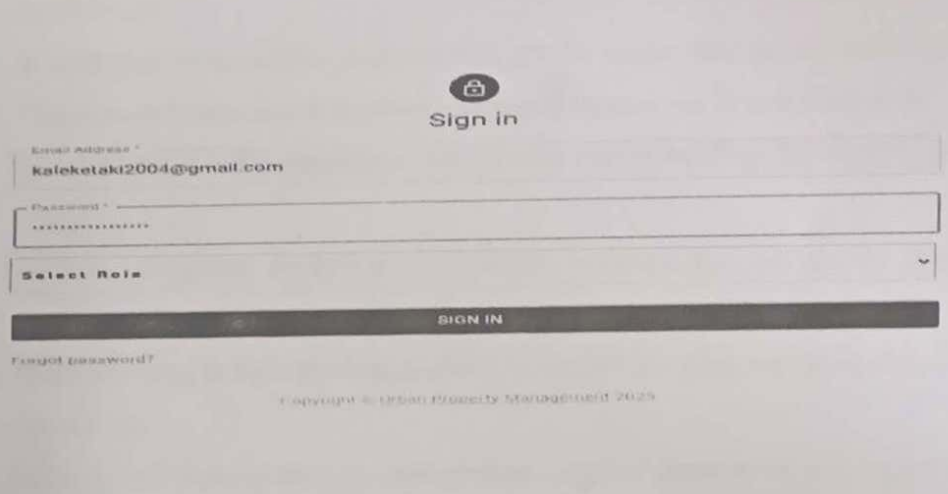
Code

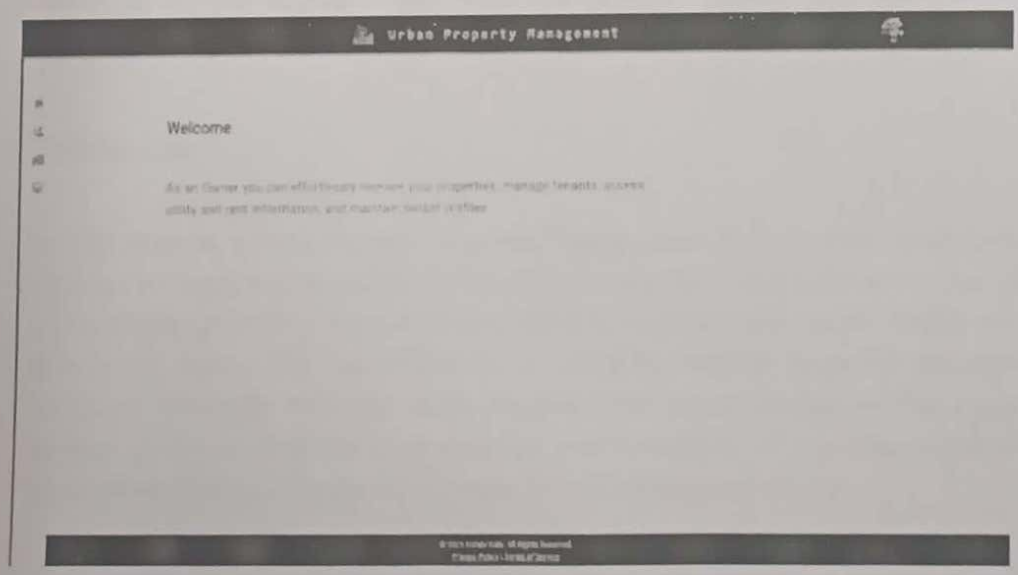
```
import * as React from 'react';
import Avatar from '@mui/material/Avatar';
import Button from '@mui/material/Button';
import CssBaseline from '@mui/material/CssBaseline';
import TextField from '@mui/material/TextField';
import Link from '@mui/material/Link';
import Paper from '@mui/material/Paper';
import Box from '@mui/material/Box';
import Grid from '@mui/material/Grid';
import LockOutlinedIcon from '@mui/icons-material/LockOutlined';
import Typography from '@mui/material/Typography';
import { createTheme, ThemeProvider } from '@mui/material/styles';
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import '../../css/Select.css'
import { [illegible] } from 'react-toastify';
functi[illegible]ght(props) {

  [illegible]ography variant="body2" color="text.secondary" align="center" {...props}>
    [illegible] Copyright © '}
    <Link[illegible]nherit" href="/">
      [illegible]perty Management
    [illegible]'}
    [illegible]ate().getFullYear()}

  [illegible]Typography>
```

```
}
const defaultTheme = createTheme();
export default function Login() {
    const [Credentials, setCredentials] = useState({ emailId: "", password: "" });
const option = ['SUPERADMIN', 'BUILDER', 'ADMIN', 'OWNER', 'TENANT'];
var [Role, SetRole] = useState("");
const onOptionChange = (event) => {
    debugger;
    SetRole(event.target.value);
  }
  const navigate = useNavigate();
  const onTextChange = (args) => {
    var copyofCredentials = { ...Credentials };
    copyofCredentials[args.target.name] = args.target.value;
    setCredentials(copyofCredentials);
    console.log(Credentials.emailId + "   " + Credentials.password);
  }
  const Validation = () => {
    if (Credentials.emailId.length == "") {
      toast.warn("Email Can Not Be Empty")
    }
    if (Credentials.password.length == "") {
      toast.warn("Password Can Not Be Empty")
    }
  }
const SignIn = () => {
    // debugger;
    Validation();
    if (Role == "SUPERADMIN") {
```

```
var helper = new XMLHttpRequest();
helper.onreadystatechange = () => {
  debugger;
  if (helper.readyState == 4 && helper.status == 200) {
    debugger
    var responseReceived = JSON.parse(helper.responseText);
    console.log(responseReceived.emailId + " " + responseReceived.password);
    debugger
    if (responseReceived.emailId == Credentials.emailId &&
responseReceived.password == Credentials.password) {
      SetRole("/SUPERADMIN");
      sessionStorage.setItem("Role", "SUPERADMIN");
      navigate("/SUPERADMIN");
      toast.success("Login Successfull");
    }
    else {
      toast.error("Invalid Credentials");
    }
  }
}
console.log(Credentials.emailId + " " + Credentials.password);
helper.open("POST", "http://localhost:7078/users/super_admin_login");
helper.setRequestHeader("Content-Type", "application/json");
helper.send(JSON.stringify(Credentials));
}
else {
  var helper = new XMLHttpRequest();
  helper.onreadystatechange = () => {
    debugger;
```

9 Screenshots of Project





10 Future Scope & Conclusion

10.1 Future Scope

1. **Online Rent & Bill Payment System** – tenants can pay rent, water & electricity bills online.
2. **Bill Generation & Receipt Module** – automated invoice generation and digital receipts.
3. **Complaint & Maintenance Request System** – tenants can submit issues & track resolution status.
4. **Notification & SMS/Email Alerts** – reminders for rent, announcements, bill alerts, etc.
5. **Visitor & Security Management** – digital visitor log, gate-entry verification for apartments.
6. **Mobile App Integration** – Android/iOS app for easier user access and notifications.
7. **Cloud-based Database & Backup** – secure data storage & real-time sync.
8. **Smart Meter & IoT Integration** – automated meter readings for electricity & water usage.
9. **AI-Based Property Analytics** – predictive maintenance, occupancy trends, cost analysis.
10. **Multi-Building & Society Management** – support for large residential complexes & townships.
11. **GIS & Govt Property Records Integration** – digital verification of property details.
12. **QR-Code Access System** – QR entry for tenants, guests, and service staff for security.

10.2 Conclusion

In this project, a basic Urban Property Management System was developed with role-based login functionality for tenants, flat owners, and managers. The system successfully provides secure authentication and limited access based on user privileges, laying the foundation for a complete digital property management solution. Although only the login modules are implemented at this stage, the project demonstrates the core concept and feasibility of creating a centralized platform to manage residential property information efficiently.